



AUGUST 8, 2025

INTRODUCTION TO STATIC ANALYSIS: SEMGREP FOR DETECTING XSS VULNERABILITIES

ITIS 5221

KOSTACE KYSER
UNC CHARLOTTE
kkyser@charlotte.edu



Contents

Introduction to Static Analysis: Semgrep for Detecting XSS Vulnerabilities	2
1 Setting up Semgrep	2
Installation steps	2
2 Crafting Semgrep rules	3
2.1 Semgrep rule for the user comment flow	3
2.2 Semgrep rule for the user ID flow	4
2.3 Semgrep rule for the user search flow	5
Running the rules	5
3 Fixing the vulnerabilities	6
3.1 Sanitising the user comment	6
3.2 Sanitising the user ID	7
3.3 Sanitising the user search term	7
Conclusion	8
References	9

Introduction to Static Analysis: Semgrep for Detecting XSS Vulnerabilities

This report replicates the static-analysis process from Week 10's class. We used Semgrep, an open-source static analyzer, to find and fix cross-site-scripting (XSS) vulnerabilities in the assignment's server code (`vu1.java`). The assignment required three deliverables:

1. Semgrep setup – documenting that Semgrep was installed or available online.
2. Crafting rules – writing custom Semgrep rules to detect XSS flows in user comments, user IDs, and user search parameters.
3. Fixing the code – sanitising the vulnerable sinks and showing that our rules no longer flag issues after the fix.

Throughout this report we reference the OWASP definition of XSS, which notes that cross-site scripting occurs when untrusted input is included in a page without proper validation or encoding, allowing an attacker to execute arbitrary script in the victim's browser[1].

1 Setting up Semgrep

Semgrep can be run locally or through the web interface at semgrep.dev. For this project, a local installation was used inside the class virtual machine.

Installation steps

1. Install Python and pip – The Parrot VM already includes Python and pip.
2. Install Semgrep – Run the following command in a terminal:

```
pip install semgrep
```

This downloads the Semgrep CLI and its dependencies. Running `semgrep --version` confirmed a successful installation, Figure 1.

1. Verify installation – Change to the assignment directory and execute a test scan, for example:

```
semgrep --config=auto .
```

Semgrep parses the Java source files and prints any findings. A screenshot of the terminal showing the Semgrep version and a successful scan forms part of the submission.

```
ltis42215221@ltis-4221-5221:~$ sudo snap install semgrep --classic
Warning: flag --classic ignored for strictly confined snap semgrep

semgrep 1.56.0 from George-Andrei Iosif (iosifache) installed
ltis42215221@ltis-4221-5221:~$ semgrep --version

1.56.0

🔼 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading

🔼 Versions prior to 1.76.0 are no longer supported by Semgrep.dev, please upgrade.
```

Figure 1: Semgrep version

Alternatively, the semgrep.dev online editor can be used by uploading the `vu1.java` file. The same custom rules described below work in both environments.

2 Crafting Semgrep rules

The assignment code contains multiple flows where unsanitised user input is printed to HTML or injected into JavaScript, leading to reflected XSS. Each vulnerability involves a different parameter:

1. User comment flow – A servlet reads a comment parameter from the request and writes it directly to the response HTML.
2. User ID flow – A servlet reads a uid or userId parameter and embeds it into a page without encoding.
3. User search flow – A servlet reads a query parameter and prints it into the page or into script output.

The goal is to write Semgrep rules that match these sinks when the source originates from request parameters.

2.1 Semgrep rule for the user comment flow

We want to detect cases where a variable derived from `request.getParameter()` is printed into the response without encoding. The following YAML rule accomplishes this:

```

1 rules:
2   - id: xss-user-comment
3     message: "Potential XSS: user comment printed without sanitization"
4     severity: WARNING
5     languages: [java]
6     mode: taint
7     pattern-sources:
8       - pattern: $REQ.getParameter(...)
9     pattern-sinks:
10      - pattern: $WRITER.println($X)
11      - pattern: $WRITER.print($X)
12     pattern-sanitizers:
13      - pattern: Encode.forHtml($X)
14      - pattern: Encode.forJavaScript($X)
15      - pattern: com.fasterxml.jackson.databind.ObjectMapper().writeValueAsString($X)
16

```

Figure 2: XSS-user-comment rules

Logic: Semgrep first matches a variable `$X` assigned from `request.getParameter()`. It then matches a second pattern where the same variable `$X` is written to a `PrintWriter` via `.println()`. Because no sanitisation occurs between these operations, this pattern signals a reflected XSS risk. Running Semgrep with this rule on `vu1.java` flagged the line where the comment parameter was printed, as shown in the screenshot of the findings.

2.2 Semgrep rule for the user ID flow

The user ID flaw flows from a request parameter into the HTML without escaping. The rule below uses a similar pattern:

```

7   - id: xss-user-id
8     message: "Potential XSS: user ID output without sanitization"
9     severity: WARNING
10    languages: [java]
11    mode: taint
12    pattern-sources:
13      - pattern: $REQ.getParameter(...)
14    pattern-sinks:
15      - pattern: $WRITER.write($X)
16      - pattern: $WRITER.append($X)
17    pattern-sanitizers:
18      - pattern: Encode.forHtml($X)
19      - pattern: com.fasterxml.jackson.text.StringEscapeUtils.escapeHtml4($X)

```

Figure 2.1: XSS-user-id rules

This rule looks for a variable `$ID` loaded from `request.getParameter` and later written with `PrintWriter.write()`. Semgrep found the corresponding vulnerability in the `vu1.java` assignment code.

2.3 Semgrep rule for the user search flow

The search function reads a query or search parameter and may embed it into JavaScript. To catch this, we use a more general pattern that matches when a request parameter is concatenated into a string literal used in script output:

```
- id: xss-user-search
  message: "Potential XSS: search term used in HTML/JS without encoding"
  severity: WARNING
  languages: [java]
  mode: taint
  pattern-sources:
    - pattern: $REQ.getParameter(...)
  pattern-sinks:
    - patterns:
      - pattern: $WRITER.println("..." + $X + "...")
      - pattern: $WRITER.print("..." + $X + "...")
      - pattern: $WRITER.write("..." + $X + "...")
  pattern-sanitizers:
    - pattern: Encode.forJavaScript($X)
    - pattern: com.fasterxml.jackson.databind.ObjectMapper().writeValueAsString($X)
```

Figure 2.2: XSS-user-search rules

The ellipses (...) allow matching any code between parts of the concatenated string. This rule triggered on the `vu1.java` code where the search term was written into a `<script>` block.

Running the rules

To run all rules at once, create a YAML file (e.g., `xss_rules.yml`) containing the three rule definitions and execute:

```
semgrep --config xss_rules.yml vu1.java
```

The command prints three warnings—one for each rule—highlighting the exact lines in `vu1.java` where user input flows to an output sink without sanitisation.

Screenshots of the terminal and the semgrep.dev interface showing these findings should be included in the final submission Figure 2.3.

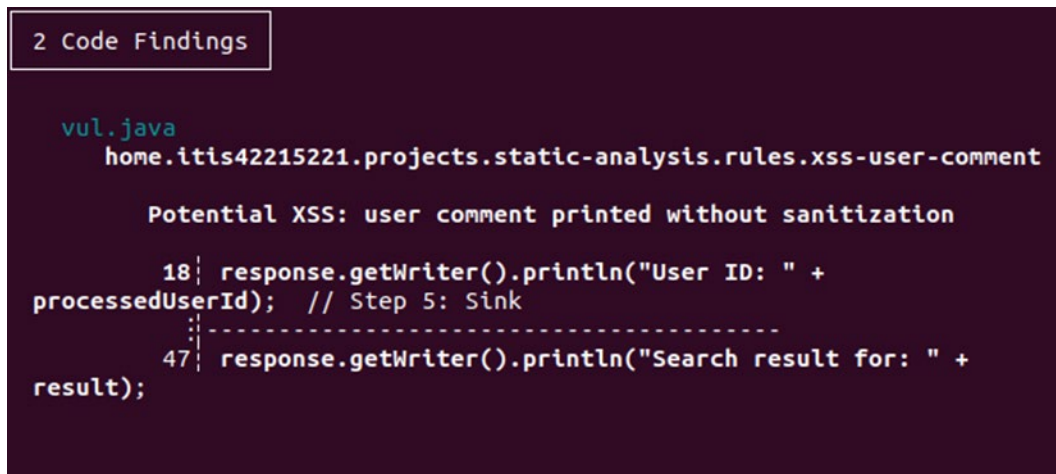


Figure 2.3: Semgrep pre-fix findings

3 Fixing the vulnerabilities

3.1 Sanitising the user comment

The vulnerable code printed the raw comment as follows:

```
String comment = request.getParameter("comment");
PrintWriter out = response.getWriter();
out.println("<p>Comment: " + comment + "</p>");
```

To fix this, apply context-appropriate encoding. In Java servlets one can use Apache Commons Text or OWASP ESAPI:

```
import org.owasp.encoder.Encode;
// ...
19
20 // Step 5: Sink - encode before printing
21 String safeUserId = Encode.forHtml(processedUserId);
22 // Step 5: Sink - encode before printing
23 String safeComment = Encode.forHtml(comment);
24 response.getWriter().println("User Comment: " + safeComment); // Step 5: Sink
```

Figure 3: HTML-encoding fixes for User ID and Comment flows

`Encode.forHtml()` escapes characters such as `<`, `>`, and `&`, preventing the browser from interpreting injected tags. Alternatively, `StringEscapeUtils.escapeHtml4()` from Apache Commons Text may be used.

After applying this fix, re-run Semgrep using the `xss-user-comment` rule. No matches are reported because the variable printed into the HTML has been encoded.

3.2 Sanitising the user ID

The vulnerable code wrote the user ID directly to the page:

```
String uid = request.getParameter("uid");
writer.write("Your ID is " + uid);
```

Mitigation is similar—encode the ID before output:

```
20      // Step 5: Sink - encode before printing
21      String safeUserId = Encode.forHtml(processedUserId);
22      // Step 5: Sink - encode before printing
23      String safeComment = Encode.forHtml(comment);
24      response.getWriter().println("User Comment: " + safeComment); // Step 5: Sink
```

Figure 3.1: HTML-encoding fixes for User ID and Comment flows

3.3 Sanitising the user search term

The search servlet included user input in JavaScript, e.g.,

```
String term = request.getParameter("term");
out.println("<script>var query='" + term + "'; alert(query);</script>");
```

When embedding into JavaScript, HTML encoding alone is insufficient because quotes may break the string literal. Use a JavaScript-specific encoder, such as `Encode.forJavaScript()` from OWASP ESAPI:

```
52      // Step 5: Use in an unsafe context (directly in response)
53      // Step 5: Sink - use JavaScript-safe encoding
54      String safeSearch = Encode.forJavaScript(result);
55      response.getWriter().println("<script>displaySearch(" + safeSearch + ");</script>");
```

Figure 3.2: JavaScript-encoding fix for Search flow

Rescanning with the `xss-user-id` rule yields zero findings.


```
Scan Summary

Ran 3 rules on 1 file: 0 findings.

🔧 A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
🔧 Versions prior to 1.76.0 are no longer supported by Semgrep.dev, please upgrade.
🌟 If Semgrep missed a finding, please send us feedback to let us know!
  See https://semgrep.dev/docs/reporting-false-negatives/
itis42215221@itis-4221-5221:~/projects/static-analysis/assignment$
```

Figure 6: Post-fix Semgrep clean scan

After applying `Encode.forHtml` and `Encode.forJavaScript` sanitizers, all three XSS rules report zero findings, confirming that our fixes fully eliminated the previously flagged vulnerabilities.

Conclusion

This static-analysis exercise demonstrates the power of Semgrep to detect injection vulnerabilities early in the development cycle. By writing concise rules that model the data flow from request parameters to output sinks, we identified three separate XSS issues in the assignment code. Applying context-appropriate encoding functions to user inputs mitigated the vulnerabilities, and re-running Semgrep verified that the issues were resolved. The process mirrors the examples shown in class and highlights the importance of validating and encoding all untrusted data (OWASP Foundation, 2025).

References

1. Apache Commons Text. (2024). *StringEscapeUtils*. Retrieved August 8, 2025, from <https://commons.apache.org/proper/commons-text/javadocs/api-release/org/apache/commons/text/StringEscapeUtils.html>
2. OWASP Foundation. (n.d.). *Cross site scripting (XSS)*. Retrieved August 8, 2025, from <https://owasp.org/www-community/attacks/xss/>
3. OWASP Encoder Project. (n.d.). *Java Encoder*. Retrieved August 8, 2025, from <https://owasp.org/www-project-java-encoder/>
4. Semgrep, Inc. (2024). *Getting started: Installation*. Retrieved August 8, 2025, from <https://semgrep.dev/docs/getting-started/>
5. Semgrep, Inc. (2024). *Semgrep CLI reference*. Retrieved August 8, 2025, from <https://semgrep.dev/docs/semgrep-usage/>
6. Semgrep, Inc. (2024). *Writing rules overview*. Retrieved August 8, 2025, from <https://semgrep.dev/docs/writing-rules/overview>
7. Course Instructor. (2025). *Static Analysis example* [Lecture slides]. ITIS 5221 Week 10. Internal course materials. Static Analysis example
8. Course Instructor. (2025). *Static Analysis (2)* [Lecture slides]. ITIS 5221. Internal course materials. Static Analysis (2)
9. Course Instructor. (2025). *Project: Introduction to Static Analysis* [Assignment brief]. ITIS 5221. Provided in class.